arXiv:1408.5456v1 [cs.LG] 23 Aug 2014

Interpreting Tree Ensembles with inTrees

Houtao Deng
California, USA

Abstract

Tree ensembles such as random forests and boosted trees are accurate but difficult to understand, debug and deploy. In this work, we provide the inTrees (interpretable trees) framework that extracts, measures, prunes and selects rules from a tree ensemble, and calculates frequent variable interactions. An rule-based learner, referred to as the simplified tree ensemble learner (STEL), can also be formed and used for future prediction. The inTrees framework can applied to both classification and regression problems, and is applicable to many types of tree ensembles, e.g., random forests, regularized random forests, and boosted trees. We implemented the inTrees algorithms in the "inTrees" R package.

Keywords: decision tree; rule extraction; rule-based learner; random forest; boosted trees.

1. Introduction

Let $X = (X_1, ..., X_p)$ be the predictor variables, T be the target (or outcome). The goal of supervised learning is to build a model using X to predict T. It is called a classification problem when T is discrete and a regression problem when T is numeric.

Tree ensembles such as random forests [3] and boosted trees [7] are accurate supervised learners, and so are powerful in capturing information in the data. However, They are difficult to a) **understand**; It may be hard, particularly for people without modeling background, to rely on a model without understanding it. b) **debug**; Just like software engineers can make bugs in programming, data analysts can make "modeling bugs" (also referred

Email address: softwaredeng@gmail.com (Houtao Deng)

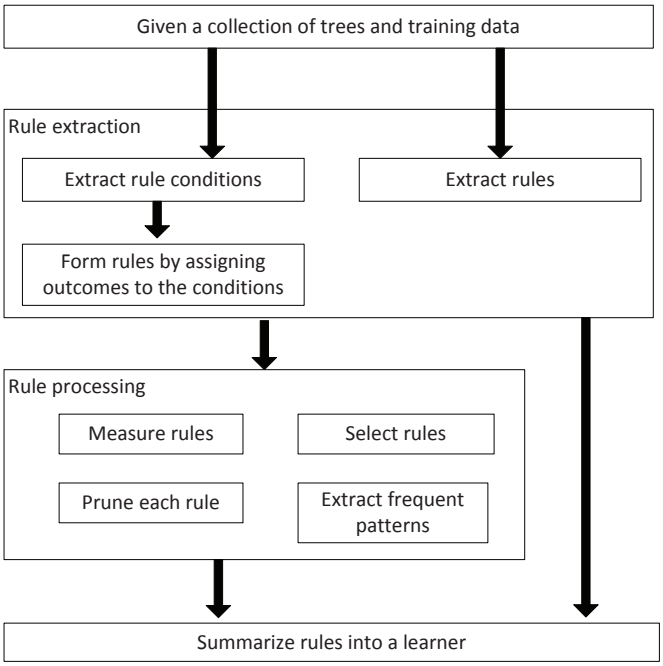


Figure 1: Illustration of the inTrees framework.

to as "leakage" in [10]), i.e., building a model in an inappropriate way so the model does not function as expected. It is hard to discover bugs in a model without interpretable information from the model. c) **deploy**. It is not easy to code a tree ensemble with potentially a huge number of trees, particularly for situations where models are trained off-line in one language such as R, but are applied on-line using another language such as Java.

In this work we propose to extract interpretable information from tree ensembles, referred to as the *inTrees* (interpretable trees) framework. Particularly, we propose methods to extract, measure and process rules from a tree ensemble, and extract frequent variable interactions. Also, the rules from a tree ensemble are used to build a rule-based learner for future predictions.

2. The inTrees Framework

We propose the inTrees (interpretable trees) framework to extract insights from tree ensembles. Particularly, inTrees consists of algorithms to extract rules, measure (and thus rank) rules, prune irrelevant or redundant variable-value pairs of a rule (prune each rule), select a compact set of relevant and

non-redundant rules (select rules), discover frequent variable interactions (extract frequent patterns), and summarize rules into a learner that can be used for predicting new data. The functions in the framework and the relationship among the functions are shown in Figure 1. One can extract rules from a tree ensemble, perform rule processing such as pruning each individual rule, and then summarize the rules into a learner. One can also directly summarize the rules after rule extraction, or just perform rule extraction and rule processing without the rule summarizing step. The inTrees framework can be applied to both classification and regression problems. Furthermore, the framework can be applied to tree ensembles with decision trees that split each internal node using one feature, and assign the outcome (or target) at leaf nodes. Many popularly used tree ensembles belong to this type, e.g., random forests [3], regularized random forests [5], and boosted trees [7].

The inTrees framework is independent from the tree ensemble building process. Therefore, as long as each tree in a tree ensemble is transformed to a specific format, inTrees algorithms can be applied. In the inTrees R implementation, the tree format is defined as the same as the "randomForest"[11] and "RRF"[5] R packages shown in Table 1. The first column in the table is the index of the current node. The next two columns are the left and right daughter nodes of the current node, respectively. Column 4 - 5, respectively, are the split variable and split value at the current node. The "status" column is "-1" for a leaf node, and other values for non-leaf nodes. The "pred" column is the prediction at a leaf node, that is, the assignment of the outcome. Note although the inTrees algorithms can be applied to a node with more than two children nodes, the inTrees R package (version 1.0) currently only handles binary splits.

Also, inTrees algorithms can be applied to each tree in a tree ensembles in parallel, thus can be implemented in distributed computing environments.

The methods in the inTrees framework are introduced in the following sections.

3. Extract Rules

A tree ensemble consists of multiple decision trees [3, 7]. A rule can be extracted from a decision tree's root node to a leaf node. A rule can be expressed as $\{C \Rightarrow T\}$, where C, referred to as the condition of the rule, is a conjunction of variable-value pairs, and T is the outcome of the rule. Algorithm 1 shows one way to extract rules from a decision tree. The rules

node	left daughter	right daughter	split var	split point	status	pred
1	2	3	5	2	1	0
2	4	5	1	3	1	0
3	6	7	3	2	1	0
4	8	9	9	3	1	0
5	10	11	2	2	1	0
6	12	13	1	2	1	0
7	14	15	7	2	1	0
8	0	0	0	0	-1	1
9	16	17	4	1	1	0
10	18	19	3	3	1	0
11	20	21	3	3	1	0
12	22	23	2	2	1	0
13	24	25	9	2	1	0
14	26	27	9	2	1	0
15	0	0	0	0	-1	2
16	28	29	7	2	1	0
17	30	31	6	2	1	0
18	32	33	7	3	1	0
19	34	35	8	1	1	0

Table 1: General tree presentation in the inTrees R package (version 1.0).

extracted from a tree ensemble are a combination of rules extracted each decision tree in the tree ensemble.

Since the outcome values of the rules from a tree ensemble are often assigned based on a part of the training data (e.g., random forest), the assignment may not be reliable. Thus, one can extract the conditions from a tree ensemble, and then assign the outcome values to the conditions based on all the training data. Algorithm 2 shows one way to extract conditions from a decision tree. Note, the most informative splits often occur in the top level of a tree, so one can stop the extraction process when a certain depth of the tree ($maxDepth$ in the algorithm) is reached, which is also more computationally efficient. To determine the outcome value of a condition, one can choose the most frequent class for a classification problem, and the mean (or other metrics such as median) for a regression problems, of the training instances that satisfy the condition.

Extracting only conditions from a tree ensemble (i.e., omitting the outcome values assigned by the tree ensemble) also has the following benefit. Say we have built a tree ensemble for regression. If one wants to derive more descriptive rules such as $\{X_1 = 1 \Rightarrow large\}$, instead of using a continuous value as the outcome, one can simply discretize the target, and then

Algorithm 1: $ruleExtract(ruleSet, node, C)$: function to extract rules $ruleSet$ from a decision tree. In the algorithm, let C denote the conjunction of variable-value pairs aggregated from the path from the root node to the current node, C_{node} denote the variable-value pair used to split the current node, $leafNode$ denote the flag whether the current node is a leaf node, and $pred_{node}$ denote the prediction at a leaf node.

input : $ruleSet \leftarrow null, node \leftarrow rootNode, C \leftarrow null$
output: $ruleSet$

if $leafNode = true$ **then**
  $currentRule \leftarrow \{C \Rightarrow pred_{node}\}$
  $ruleSet \leftarrow \{ruleSet, currentRule\}$
  return $ruleSet$
end
for $child_i$ = *every child of node* **do**
  $C \leftarrow C \wedge C_{node}$
  $ruleSet \leftarrow ruleExtract(ruleSet, child_i, C)$
end
return $ruleSet$

assign discretized levels to the conditions (same as assigning the outcome to a condition for classification problems).

4. Measure rules

Here we introduce metrics to measure rules' quality. **Frequency** of a rule is defined as the proportion of data instances satisfying the rule condition. The frequency measures the popularity of the rule. **Error** of a rule is defined as the number of incorrectly classified instances determined by the rule divided by the number of instances satisfying the rule condition for classification problems, and mean squared error for regression problems,

$$MSE = \frac{1}{k}\sum_{i=1}^{k}(t_i - \bar{t})^2 \tag{1}$$

where k is the number of instances that satisfy the rule condition, and t_i (i=1,...,k) are the target values of the instances satisfying the condition, and $\bar{t} = \frac{1}{k}\sum_{i=1}^{k}(t_i)$.

Complexity of a rule is measured by the **length** of the rule condition, defined as the number of variable-value pairs in the condition. Given two rules with similar frequency and error, the rule with a smaller length may be preferred as it is more interpretable.

Algorithm 2: $condExtract(condSet, node, C, maxDepth, curretDepth)$: function to extract conditions *condSet* from a tree ensemble. In the algorithm, let C denote the conjunction of variable-value pairs aggregated in the path from the root node to the current node, C_{node} denote the variable-value pair used to split the current node, $leafNode$ denote the flag whether the current node is a leaf node. Note one can set a maximum depth $maxDepth$ of the tree where the conditions are extracted from. In a decision tree, most useful splits tend to happen in top levels of the trees (i.e., when *depth* is small), so setting a maximum depth can reduce computations, and may also avoid extracting overfitting rules. $maxDepth$=-1 means there is no limitation on the depth.

input : $condSet \leftarrow null, node \leftarrow rootNode, C \leftarrow null$, $maxDepth \leftarrow -1, curretDepth \leftarrow 0$
output: *condSet*

$curretDepth = curretDepth + 1$
if $leafNode = true$ *or* $curretDepth = maxDepth$ **then**
  $condSet \leftarrow \{condSet, currentCond\}$
  return *condSet*
end
for $child_i$ = *every child of node* **do**
  $C \leftarrow C \wedge C_{node}$
  $condSet \leftarrow condExtract(condSet, child_i, C, maxDepth, curretDepth)$
end
return *condSet*

By using these definitions, one can rank the rules according to length, support, error or a combination of multiple metrics.

5. Prune rules

The condition of a rule $\{C \Rightarrow T\}$ consists of variable-value pairs, $a_1 = v_1$, ..., $a_i = v_i$, ..., $a_K = v_K$, where a_i and v_i is the ith variable-value pair in the condition, and K ($K \geq 1$) is the total number of variable-value pairs in the condition. A rule extracted from trees may include irrelevant variable-value pairs. This section provides methods to prune irrelevant variable-value pairs from rule conditions.

Let E denote a metric measuring the quality of a rule, and a smaller value of E indicates a better rule. Examples of E include error of applying a rule to the training data, error of applying a rule to a validation data set, and pessimistic error that is a combination of training error and model complexity. Here we assume $E \geq 0$.

Let E_0 denote the E of the original rule $\{C \Rightarrow T\}$, and let E_{-i} denote the E of the rule that leave the ith variable-value pair out. We use $decay_i$ to

evaluate the effect of removing the ith pair:

$$decay_i = \frac{E_{-i} - E_0}{max(E_0, s)} \tag{2}$$

where s is a positive number that bound the value of $decay_i$ when E_0 is 0 or very small. In the inTree R package, we currently set $s = 10^{-6}$.

Equation 2 can be interpreted as the **relative** increase of error after removing a variable-value pair. Alternatively, one can define the decay function as the increase of error shown in Equation 3.

$$decay_i = E_{-i} - E_0 \tag{3}$$

When $decay_i$ is smaller than a threshold, e.g., 0.05, the ith variable-value pair may be considered unimportant for the rule and thus can be removed. We call this pruning method **leave-one-out pruning**.

Leave-one-out pruning is applied to each variable-value pair sequentially. Consider example: $\{X_1 = 0 \ \& \ X_2 = 1 \ \& \ X_3 = 0 \Rightarrow true\}$. Use Equation 2 as the decay function, and set s as 0.001 and the decay threshold as 0.05. Start with the last variable-value pair, assuming $E_0 = 0.1$, $E_{-3} = 0.2$, then $decay_3 = \frac{0.2-0.1}{0.1} = 1$. As $decay_3 > 0.05$, the last pair remains in the rule, and thus E_0 remains 0.1. Next consider the second pair $X_2 = 1$. Assuming $E_{-2} = 0.104$, and thus $decay_2 = \frac{0.104-0.1}{0.1} = 0.04 < 0.05$, then $X_2 = 1$ is removed from the rule. Now the rule becomes $\{X_1 = 0 \ \& \ X_3 = 0 \Rightarrow true\}$, and $E_0 = 0.104$. Finally consider the first pair $X_1 = 0$ in $\{X_1 = 0 \ \& \ X_3 = 0 \Rightarrow true\}$. Assuming $E_{-1} = 0.3$, then $decay_1 = \frac{0.3-0.104}{0.104} > 0.05$, and thus the first pair remains in the rule. And the rule after pruning is $\{X_1 = 0 \ \& \ X_3 = 0 \Rightarrow true\}$. In this work we don't optimize the order of the pairs to be considered for pruning.

6. Select Rules

6.1. Rule selection via feature selection

The number of rules extracted from a tree ensemble can be large. We can rank the rules by error and frequency, however, the top rules could be similar to each other, i.e., redundant. It would be desirable to derive a compact rule set that contains relevant and non-redundant rules. [6] uses feature selection to select a set of relevant and non-redundant conditions from associative classification rules [12]. Here we apply feature selection to the conditions

from a tree ensemble. The idea is creating a new data set I by the following way described in [6].

Let $\{c_1, c_2, ..., c_J\}$ denote the condition set of a rule set. Let I_{ij} denote whether a condition c_j is satisfied by the ith instance, that is,

$$I_{ij} = \begin{cases} 1 & c_j \text{ is satisfied for the } i^{th} \text{ instance} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

A new data set I is formed by combining the binary variables and the target: $\{[I_{i1}, ..., I_{iJ}, t_i], i = 1, ...n\}$ where t_i is the target value of the ith instance, and n is the number of instances for training. Let $\{I_1, I_2, ..., I_J\}$ denote the predictor variables. Then feature selection can be applied to the new data set to select a set of relevant and non-redundant variables, each essentially presenting a condition. More details of the algorithm can be found in [6]. Furthermore, once conditions is selected, one can use the approach introduced in Section 3 to assign outcomes to the conditions.

6.2. Complexity-guided condition selection

In [6] the condition complexity (length of a condition) was not considered in the feature selection process. Here we consider condition complexity in the feature selection process by using the guided regularized random forest (GRRF) algorithm [5].

In a regularized random forest (RRF), the regularized information gain in a tree node is defined as

$$Gain_R(X_i) = \begin{cases} \lambda_i \cdot Gain(X_i) & X_i \notin F \\ Gain(X_i) & X_i \in F \end{cases} \tag{5}$$

where F is the set of indices of variables used for splitting the previous nodes and is an empty set at the root node in the first tree. $Gain(\cdot)$ represents an ordinary information gain metric in decision trees. $\lambda_i \in (0, 1]$ is called the penalty coefficient. The regularized information gain penalizes the ith variable when $i \notin F$, that is, the ith variable was not used in previous nodes. The regularized random forest adds the index of a new variable to F if the variable adds enough new predictive information to the already selected variables.

A smaller λ_i leads to a larger penalty. In RRF, λ_i is the same for all variables. GRRF [5] assigns a value to λ_i based on the global importance

score of X_i calculated from an ordinary random forest (RF). A larger importance score of X_i would lead to a larger λ_i. Therefore, given two variables that are not selected in previous nodes and have similar information gain, the variable with a larger global importance score has a larger regularized information gain.

Here we further extend the concept of GRRF to handle rule complexity. Particularly, $\lambda_i \in (0, 1]$ is calculated based on the length of the condition represented by i^{th} variable. That is,

$$\lambda_i = \lambda_0 \cdot (1 - \gamma \ \cdot \frac{l_i}{l^*}) \tag{6}$$

where $\lambda_0 \in (0, 1]$ is called the base coefficient, l_i is the length of the condition represented by the ith variable in I, l^* is the maximum length of the conditions in the condition set, and $\gamma \in [0, 1]$ controls the weight of the regularization on condition length. It can be seen a longer rule condition leads to a smaller λ_i, and thus is penalized more in Equation 5. In addition, it may be also useful to add the global importance score to Equation 6 as follows

$$\lambda_i = \lambda_0 \cdot (1 - \gamma \ \cdot \frac{l_i}{l^*} + \beta \cdot imp_i) \tag{7}$$

where imp_i is the normalized importance score ($0 \leq imp_i \leq 1$) of X_i, calculated from an ordinary RF, and $\beta \in [0, 1]$ controls the weight of the global importance scores.

Also, GRRF not only can select a subset of variables, but can also provide a score for each variable. However, the score can be biased in favour of variables entering the variable subset F earlier in the feature selection process. One can build an ordinary RF on the selected variables, to calculate scores for the selected conditions.

7. Variable interaction extraction based on association rule analysis

Here we introduce methods to extract frequent variable interactions in a tree ensemble using association rule analysis [1].

Association rule analysis [1, 9] discovers associations between items in transaction data sets, where one transaction consists of one or multiple items. An example of an association rule is $\{bread, butter \Rightarrow milk\}$, that is, transactions contain bread and butter also contain milk. The left-hand side of a

rule is called the condition, and the right-hand side is called the outcome. An association rule can be measured by the following three metrics. **Support** is defined as the proportion of transactions containing the condition. **Confidence** is defined as the number of transactions containing both the condition and the outcome divided by the number of transactions containing the condition. **Length** is defined as the number of items contained in an association rule. For example, $\{bread, butter \Rightarrow milk\}$ has a length of 3.

Each rule from tree ensemble has the form of $C \Rightarrow T_k$ consists of variable-value pairs and the target-value pair, i.e., $a_1 = v_1$, ..., $a_i = v_i$, ..., $a_K = v_K$ and $T = T_k$. One can treat each variable-value pair $a_i = v_i$ or target-value pair $T = T_k$ as an item. Association rule analysis can then be used to obtain a set of association rules with a minimum support and confidence, but with a restriction that the left-hand side of the association rule contains only variable-value pairs, and the right-hand side contains only the target-value pair. Now one can rank the association rules by any one of metrics: support, confidence, length, or a combination of multiple metrics. Note the variable-value pairs contained in the association rule(s) with the largest support and length l ($l \geq 2$) can be considered as the most frequent l-way variable interactions.

The concept of "item" in association rule can be used flexibly. For example, the variable of an variable-value pair can also be an item. This can be useful for the following situation. Numeric variables can have many combinations of variable-value pairs in the rule sets extracted from a tree ensemble. If we consider variable-values or target-value pairs as items, the support of association rules containing the variable-value pair of the numeric variables can be relatively small. To solve this issue, one can discretize the numeric variables. Alternatively, one can treat numeric variables, and variable-value pairs for discrete variables as items.

8. Simplified tree ensemble learner (STEL)

The rules extracted from a tree ensemble can be summarized into a rule-based learner, referred to as a simplified tree ensemble learner (STEL). [8] used a linear model summarizing the rules from a tree ensemble. Also, ideas in summarize associative classification rules into classifiers [12, 6] could be applied to tree ensemble rules. Here we introduce one algorithm to summarize rules.

Let $\Re$ denote an ordered rule list, and $\Re = \{\}$ at the beginning of the algorithm. The goal is to build a list of rule in $\Re$ ordered by priority. Then the rules in $\Re$ are applied, from the top to the bottom, to a new instance until a rule condition is satisfied by the instance. The right-hand side (i.e., outcome) of the rule then becomes the prediction for the new instance.

First define the default rule as $\{\} \Rightarrow t^*$, where t^* is the most frequent class in the training data for classification, and the mean for regression. The default rule assigns t^* to an instance's outcome disregarding the predictor variable values of the instance.

Let S denote the rule set including the two default rules and rules extracted from a tree ensemble. Rules below a certain frequency, e.g., 0.01, are removed from S to avoid overfitting. Let D denote the set including all the training instances at the start of the algorithm.

The algorithm consists of multiple iterations. At each iteration, the best rule in S evaluated by data D is selected and added to the end of $\Re$. The best rule is defined as the rule with the minimum error. If there are ties, the rule with higher frequency is preferred. Preference on rules with shorter conditions is used to break further ties. Then the data instances satisfying the best rule are removed from D, and the default rule and the rule metrics are updated based on the data instances left. If the default rule is selected as the best rule in an iteration, the algorithm returns $\Re$. If no instance is left, the default rule is assigned with the most frequent class for classification or mean for regression in the original training data. The default rule is added to the end of $\Re$, and the algorithm returns $\Re$.

Algorithms in summarizing associative classification rules may be useful in summarizing the rules from tree ensembles. For example, one can apply a decision tree algorithm to the new data set I, which summarizes the conditions into a rule-based classifier [6]. But note the rules from tree ensembles are different than associative classification rules in that a variable or target in a tree ensemble rule can be numeric or discrete, while associative classification rules only handle discrete variables and target.

9. Transform regression rules to classification rules

A regression rule with an numeric outcome, e.g., $\{X_1 = 1 \Rightarrow 100\}$, may not be as interpretable as classification rules, e.g., $\{X_1 = 1 \Rightarrow large\}$. In the inTrees framework, one can easily transform regression rules to classification rules as follows. Firstly, build a tree ensemble for a regression problem, and

extract rule conditions from the tree ensemble. Secondly, replace the numeric target variable with a discretized version (e.g., discretization with equal frequency). Then the methods dealing with classification rules can be used for the conditions extracted. Therefore, one can transform the regression rules to classification rules without re-building a tree ensemble after discretizing the target variable.

10. Illustrative Examples

Here we illustrate the inTrees functions by examples. Consider the following team optimization problem. One chooses a team with 10 players from 20 to play a game against another team. The team would win if a) either player 1 or player 2 is in the team; and b) Player 1 and player 2 do not play together in the team. Otherwise, the team would lose. The logic is shown in table 2.

Table 2: The team optimization example where the team would win only if either player 1 or player 2 plays, and they do not play together. In the table "Y" stands for playing in the team, and "N" stands for not playing.

player 1	player 2	player 3	...	outcome
N	N	...	...	lose
Y	Y	...	...	lose
Y	N	...	...	win
N	Y	...	...	win

We simulated the data set with 20 predictive variables X_1, ..., X_{20}, the target T, and 100 instances (X_i represents player i). For each instance, 10 variables are randomly selected and assigned value "Y", and other variables are assigned value "N". T = "win" when one and only one of X_1 and X_2 equals "Y", and T = "lose" otherwise.

In the following we apply the inTrees framework to the data set. We build a regularized random forest (RRF) with 100 trees on the data set. 1923 rule conditions with maximum length of 6 were extracted from the RRF by the condition extraction method in the inTrees package. After de-duping the same rules, there are 1835 unique rule conditions.

Next, inTrees provides functions to assign outcomes to the conditions, and, in the mean time, calculates the length, frequency and error of each rule. Alternatively, one can extract rules from a random forest using algorithm 1.

However, the original outcome assignments of the rules from RF may not be reliable because each rule is evaluated by a sub-sample of training data.

Table 3 shows two examples of rules consisting conditions and predictions (outcome assignments), and the rule metrics: length, frequency, and error. In the table "X[,k] %in% c(v)" represents that the kth variable equals one of the values from set v when $X[,k]$ is categorical. When a variable is numeric, "$\leq$" or "$\geq$" is used as the relation operator between the variable and a numeric value. The conditions are presented in an R language format, and thus can be directly executed in R given "X" is the predictor variable matrix (return "TRUE" for instances satisfying the conditions, and "FALSE" for other instances).

Table 3: Two examples of rules consisting of conditions and predictions (denoted by "pred"), and the metrics of rules.

len	freq	err	condition	pred
3	0.07	0	X[,1] %in% c('N') & X[,2] %in% c('N') & X[,19] %in% c('N')	lose
3	0.16	0	X[,1] %in% c('Y') & X[,2] %in% c('N') & X[,19] %in% c('N')	win

Next discuss pruning each rule. The rules shown in Table 3 after being pruned have shorter conditions shown in Table 4. With the irrelevant variable-value pairs being removed from the conditions, the frequency of the both rules increases without increase of error.

Table 4: Pruned rules for the rules shown in Table 3.

len	freq	err	condition	pred
2	0.22	0	X[,1] %in% c('N') & X[,2] %in% c('N')	lose
2	0.24	0	X[,1] %in% c('Y') & X[,2] %in% c('N')	win

Now consider rule selection. We applied the complexity-guided regularized random forest to the rule set (with each rule being pruned). Table 5 shows the selected rules and their metrics.

Table 5: Rule set selected by guided regularized random forest.

len	freq	err	condition	pred
2	0.22	0	X[,1] %in% c('N') & X[,2] %in% c('N')	lose
2	0.22	0	X[,1] %in% c('Y') & X[,2] %in% c('Y')	lose
2	0.24	0	X[,1] %in% c('Y') & X[,2] %in% c('N')	win
2	0.32	0	X[,1] %in% c('N') & X[,2] %in% c('Y')	win

Furthermore, one can build an ordered list of rules that can be used as a classifier, i.e., the simplified tree ensemble learner (STEL). Table 6 shows

the classifier. A more readable version of the classifier is presented in Table 7.

Table 6: An ordered list of rules as a classifier. Note the frequency (denoted by "freq") and error of (denoted by "err") of a rule in the classifier are calculated at the iteration the rule is selected, and so can be different than applying the rules individually to the whole training data. These conditions are R-executable given X is the data including the predictive variables. The last rule "X[,1]==X[,1]" with prediction of TRUE simply means "All instances that do not satisfy the above rules are predicted as lose".

len	freq	err	condition	pred
2	0.32	0	X[,1] %in% c('N') & X[,2] %in% c('Y')	win
2	0.24	0	X[,1] %in% c('Y') & X[,2] %in% c('N')	win
1	0.44	0	X[,1]==X[,1]	lose

Table 7: Present STEL rules in a more readable form.

len	freq	err	condition	pred
2	0.32	0	X1 %in% c('N') & X2 %in% c('Y')	win
2	0.24	0	X1 %in% c('Y') & X2 %in% c('N')	win
1	0.44	0	Else	lose

One can also use inTrees to extract frequent variable interactions from the rules extracted from a tree ensemble (without removing identical rules, pruning rules or selecting rules). The top 10 most frequent combinations of variable-value pairs (length ≥ 2) are shown in the "condition" column of table 8, as a combination of variable-value pairs is essentially a condition. It can be seen that the top 4 conditions capture the true patterns. About 4% (support) of the rules from the random forest contain each of the top 4 conditions, and 100% of the rules containing one of the top 4 conditions have the true outcomes (confidence = 1). The fifth and sixth conditions in the table have support more than 3% but confidence less than 70%. Therefore these frequent variable interactions do not lead to consistent outcome assignments. This indicates one should look at both the frequency and the confidence when considering variable interactions.

11. Experiments

To test the accuracy of the simplified tree ensemble learner (STEL) implemented in the "inTrees" package version 1.0, we compare it to the popularly used decision tree method in the rpart package [4, 14] on 20 data sets from UCI[2]. Random forest with 100 trees was used. Conditions with maximum

Table 8: Top 10 most frequent combinations of variable-value pairs with length ≥ 2 appearing in the tree ensemble, and their metrics. Column "len" measures the length of the conditions (or variable interactions)

len	sup	conf	condition	pred
2	0.046	1	X[,1] %in% c('N') & X[,2] %in% c('N')	lose
2	0.044	1	X[,1] %in% c('Y') & X[,2] %in% c('N')	win
2	0.041	1	X[,1] %in% c('N') & X[,2] %in% c('Y')	win
2	0.039	1	X[,1] %in% c('Y') & X[,2] %in% c('Y')	lose
2	0.034	0.699	X[,12] %in% c('N') & X[,5] %in% c('Y')	win
2	0.032	0.667	X[,12] %in% c('Y') & X[,19] %in% c('Y')	lose
2	0.029	0.696	X[,11] %in% c('Y') & X[,19] %in% c('Y')	lose
2	0.028	0.635	X[,11] %in% c('Y') & X[,12] %in% c('Y')	lose
2	0.026	0.595	X[,12] %in% c('N') & X[,5] %in% c('N')	lose
2	0.025	0.615	X[,19] %in% c('Y') & X[,9] %in% c('Y')	lose

length of 6 were extracted from random forest. Also, for data sets with more than 2000 conditions extracted, 2000 conditions were randomly sampled and used. For each data set, we randomly sampled 2/3 of the data for training the models, and used the rest 1/3 for testing the models. The procedure was performed 100 times (each run is different due to randomness). The average error rate for each method and the relative difference between the two methods (defined as the difference between the larger error and lower error divided by the larger error) on each data set were calculated. The paired t-test was also applied to the error rates of the two methods from the 100 runs for each data set. The data sets with a significant difference between the two methods at the 0.05 level are marked with "$\circ$" (STEL worse than rpart) or "$\bullet$" (STEL better than rpart) in the table.

The results are shown in table 9. STEL outperforms rpart with a statistically significant difference for 13 data sets, and loses with a statistically significant difference for only 5 data sets. Furthermore, when STEL outperforms rpart, most of the relative differences are greater than 10%, while when STEL loses, only one data has a relative difference more than 10% (16.6%).

Table 10 shows the most accurate rule that has a frequency greater than 0.1 for each data set. Most rules have error rate of 0. The rule for "led7" has error rate greater than 0.2 (i.e., 20%), which may be because the data set is difficult to classify (Table 9 shows it indeed has relatively high error rate).

12. Conclusions

In this work we propose the inTree framework that consists of algorithms extracting, measuring, pruning, and selecting conditions/rules, and extract-

Table 9: Summary of 20 data sets, and error rates of rpart and the simplified tree ensemble learner (STEL) for each data set. Relative differences are calculated. The data sets with a significant difference at the 0.05 level are marked with "$\circ$" when STEL has higher error rates than rpart or "$\bullet$" when STEL has lower error rates. STEL outperforms rpart for 13 data sets but only loses for 5 data sets (with a statistically significant difference). Furthermore, the relative differences tend to be larger when STEL has a advantage.

	numInst	numFea	rpart	STEL	difference(%)
anneal	898	38	0.098	0.070 $\bullet$	28.7%
austra	690	14	0.145	0.157 $\circ$	8.0%
auto	205	25	0.376	0.262 $\bullet$	30.2%
breast	699	10	0.058	0.048 $\bullet$	17.4%
crx	690	15	0.148	0.159 $\circ$	7.2%
german	1000	20	0.274	0.286 $\circ$	4.3%
glass	214	9	0.342	0.310 $\bullet$	9.2%
heart	270	13	0.219	0.224	2.1%
hepati	155	19	0.209	0.211	0.6%
horse	368	22	0.164	0.197 $\circ$	16.6%
iris	150	4	0.064	0.047 $\bullet$	26.6%
labor	57	16	0.223	0.148 $\bullet$	33.7%
led7	3200	7	0.318	0.269 $\bullet$	15.3%
lymph	148	18	0.268	0.209 $\bullet$	21.9%
pima	768	8	0.260	0.272 $\circ$	4.4%
tic-tac	958	9	0.094	0.002 $\bullet$	97.9%
vehicle	846	18	0.325	0.285 $\bullet$	12.2%
waveform	5000	21	0.262	0.198 $\bullet$	24.2%
wine	178	13	0.122	0.086 $\bullet$	29.8%
zoo	101	16	0.211	0.061 $\bullet$	71.3%

ing frequent variable interactions/conditions from tree ensembles. Note the methods here can be applied to both classification and regression problems. The inTrees framework has been implemented in the "inTrees" R package for tree ensembles: random forests, regularized random forests, and the generalized boosted regression models included in the "gbm" R package [13].

This work also demonstrates that the rules from tree ensembles can be accurate after being processed. This conclusion can be valuable to the rule mining area. For example, in the associative classification rule mining area (leveraging association rule [1] for classification), numerous algorithms have been proposed to select a subset of associative classification rules or summarize rules into a classifier [12, 6]. However, the predictor variables and the target need to be discrete (thus only for classification problems), and extracting associative classification rules can be computationally expensive. On the other hand, tree ensembles can be built and the rules can be extracted efficiently (also, both tree building and rule extraction can be performed in a distributed manner). Furthermore, tree ensembles can be built on mixed categorical and numeric predictors, and can be used for both classification

Table 10: The most accurate rule with a minimum frequency of 0.1 for each UCI data set. "bwnfp" for "glass" data set stands for class "building_windows_non_float_processed".

	len	freq	err	condition	pred
anneal	5	0.342	0	X4<=1.5 & X5<=82.5 & X7 %in% c('S') & X8<=2.5 & X33<=0.7995	3
austra	5	0.181	0	X5<=7.5 & X7<=3.375 & X8<=0.5 & X13<=415.5 & X14<=251	0
auto	5	0.195	0	X1>71.5 & X2 %in% c('bmw','honda','isuzu', 'jaguar','mazda','nissan','peugot','subaru','toyota') & X5 %in% c('four') & X10<=187.25 & X21>69.5	0
breast	3	0.591	0	X3<=3.5 & X7<=2.5 & X9<=3.5	benign
crx	4	0.188	0	X3>1.5625 & X6 %in% c('aa','c','d','ff','i','j', 'k','m','r') & X9 %in% c('f') & X15<=492	no
german	5	0.132	0.015	X1 %in% c('no-account') & X5<=4103.5 & X10 %in% c(' guarantor',' none') & X13>33.5 & X14 %in% c(' none')	good
glass	5	0.136	0	X1<=1.517325 & X3>2.7 & X4>1.42 & X7>7.82 & X9<=0.16	bwnfp
heart	4	0.2	0	X1<=55.5 & X4>119 & X10<=1.7 & X13<=4.5	1
hepati	6	0.542	0	X1<=61.5 & X11>1.5 & X13>1.5 & X14<=3.7 & X15<=218.5 & X18>40.5	2
horse	5	0.188	0	X3<=38.45 & X3>37.25 & X4<=126 & X10>2.5 & X12>2.5	1
iris	1	0.333	0	X3<=2.55	setosa
labor	4	0.614	0	X2>2.75 & X7 %in% c('empl_contr','ret_allw') & X8>5 & X13 %in% c('yes')	good
led7	3	0.103	0.211	X1<=0.5 & X2<=0.5 & X6>0.5	1
lymph	4	0.284	0	X2>1.5 & X13>2.5 & X13<=3.5 & X18<=2.5	2
pima	3	0.124	0	X2<=106.5 & X6<=29.95 & X8<=28.5	0
tic-tac	3	0.225	0	X1 %in% c('b','x') & X5 %in% c('b','x') & X9 %in% c('b','x')	positive
vehicle	5	0.102	0	X3>71.5 & X6>8.5 & X7>142.5 & X12<=376.5 & X14>63.5	4
waveform	5	0.102	0.059	X6<=1.655 & X9<=2.99 & X11>3.415 & X12>2.49 & X14>2.075	2
wine	3	0.337	0	X1<=12.78 & X2<=4.575 & X10<=4.84	2
zoo	1	0.406	0	X4>0.5	1

and regression problems. Note the ideas selecting and summarizing associative classification rules could still be applied to the rules extracted from tree ensembles.

References

[1] R. Agrawal, R. Srikant, et al., Fast algorithms for mining association rules, in: Proc. 20th int. conf. very large data bases, VLDB, volume 1215, pp. 487–499.

[2] C. Blake, C. Merz, UCI repository of machine learning databases (1998).

[3] L. Breiman, Random forests, Machine learning 45 (2001) 5–32.

[4] L. Breiman, J. Friedman, R. Olshen, C. Stone., Classification and Regression Trees, Wadsworth, Belmont, MA, 1984.

[5] H. Deng, G. Runger, Gene selection with guided regularized random forest, Pattern Recognition 46 (2013) 3483–3489.

[6] H. Deng, G. Runger, E. Tuv, W. Bannister, Cbc: An associative classifier with a small number of rules, Decision Support Systems 59 (2014) 163–170.

[7] J.H. Friedman, Greedy function approximation: a gradient boosting machine, Annals of Statistics (2001) 1189–1232.

[8] J.H. Friedman, B.E. Popescu, Predictive learning via rule ensembles, The Annals of Applied Statistics (2008) 916–954.

[9] M. Hahsler, B. Grün, K. Hornik, Introduction to arules–mining association rules and frequent item sets, SIGKDD EXPLORATIONS (2007).

[10] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data (TKDD) 6 (2012) 15.

[11] A. Liaw, M. Wiener, Classification and regression by randomforest, R news 2 (2002) 18–22.

[12] B. Liu, W. Hsu, Y. Ma, Integrating classification and association rule mining, in: Proceeding of the 1998 International Conference on Knowledge Discovery and Data Mining, ACM, 1998, pp. 80–86.

[13] G. Ridgeway, Generalized boosted regression models, Documentation on the R Package gbm, version 1· 5 7 (2006).

[14] T.M. Therneau, B. Atkinson, B. Ripley, rpart: Recursive partitioning, R package version 3 (2010).